

# Real-Time Schedulers for STM32: Performance Evaluation of EDF over SuperLoop

Vladimir Toporkov, Artem Burmyakov  
Innopolis University

ToDo

1. AB: To introduce one paragraph per each point: . . .
2. AB: Can we spell “SuperLoop” instead of “Super Loop”? . . . . .
3. AB: to create a .bib file and add reference . . . . .
4. AB: to put reference to Burmyakov2026. . . . .

## I. INTRODUCTION

1.AB: To introduce one paragraph per each point:

- 1) STM32 microcontroller;
- 2) Tasks from peripheral sensors;
- 3) Real-time scheduling: SuperLoop and EDF.

2.AB: Can we spell “SuperLoop” instead of “Super Loop”?

### A. Contribution

We evaluate the scheduling performance of EDF over the default Super Loop scheduler for STM32 systems. Our implementations for both schedulers are publicly available.

## II. BACKGROUND ON STM32

We illustrate the hardware organization for our system in Fig. 1. The central processing unit is an STM32F767 uniprocessor, which in turn is connected to four sensors:

- DHT11: to measure temperature and humidity;
- HC-SR04: to measure distance;
- OV2640: to capture images;
- SPW2430: to acquire acoustic signal.

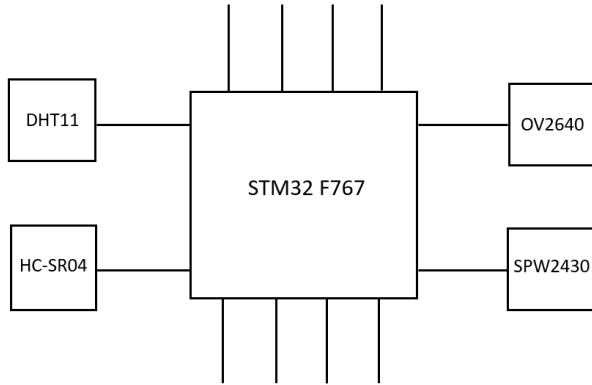


Fig. 1. Our experimental hardware platform.

## III. SYSTEM MODEL

**P.** We rely on a classical real-time periodic system model by Liu and Layland **3.AB: to create a .bib file and add reference.** We adopt the task and release notation of Burmyakov and Bini from **4.AB: to put reference to Burmyakov2026.**

A real-time system  $\mathcal{T} = \{\tau_1, \dots, \tau_n\}$  consists of  $n$  periodic tasks executed on a uniprocessor. Each task  $\tau_i = (C_i, D_i, P_i)$  has a worst-case execution time  $C_i$ , a relative deadline  $D_i$ , and a period  $P_i$ . We consider implicit deadlines,  $D_i = P_i$ . Task utilization is  $U_i = C_i/P_i$ , and task-set utilization is  $U_{\mathcal{T}} = \sum_{\tau_i \in \mathcal{T}} U_i$ .

Let  $J_{i,j}$  denote the  $j$ -th job of  $\tau_i$ , released at  $r_{i,j}$ . Unlike the sporadic model in the referenced paper, consecutive releases are separated by exactly  $P_i$ :

$$r_{i,j+1} = r_{i,j} + P_i. \quad (1)$$

Job  $J_{i,j}$  requires at most  $C_i$  units of CPU time and has an absolute deadline  $r_{i,j} + D_i$ . Its completion time, when executed to completion, is denoted by  $f_{i,j}$ . Release indices include jobs skipped by the implementation; these have no completion time.

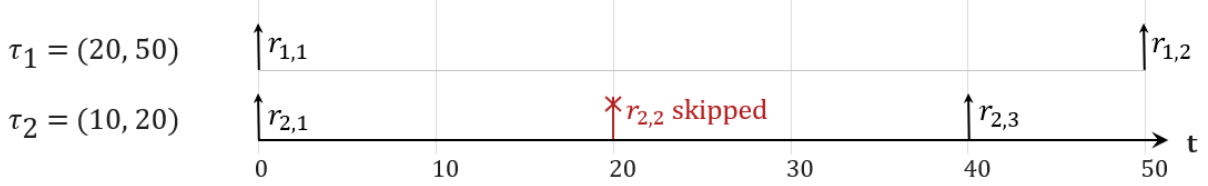
- A schedule specifies which job executes on the processor at each time instant, or whether the processor executes no job.
- Preemption interrupts a job before completion to execute another job. The interrupted job may resume later.
- A completed job  $J_{i,j}$  has missed its deadline if  $f_{i,j} > r_{i,j} + D_i$ .
- The response time of a completed job  $J_{i,j}$  is  $f_{i,j} - r_{i,j}$ , the time from its release to completion.

TABLE I  
SYSTEM MODEL NOTATION.

| Symbol                                                | Meaning                                        |
|-------------------------------------------------------|------------------------------------------------|
| $\mathcal{T}, n$                                      | Task set and number of tasks                   |
| $\tau_i$                                              | Task $i$                                       |
| $C_i$                                                 | Worst-case execution time of a job of $\tau_i$ |
| $P_i$                                                 | Task period                                    |
| $D_i = P_i$                                           | Relative deadline, equal to the period         |
| $J_{i,j}$                                             | The $j$ -th job of task $\tau_i$               |
| $r_{i,j}$                                             | Release time of job $J_{i,j}$                  |
| $f_{i,j}$                                             | Completion time of an executed job             |
| $r_{i,j} + D_i$                                       | Absolute deadline of job $J_{i,j}$             |
| $U_i = C_i/P_i$                                       | Task utilization                               |
| $U_{\mathcal{T}} = \sum_{\tau_i \in \mathcal{T}} U_i$ | Task-set utilization                           |

$$\tau_i = (C_i, D_i, P_i), D_i = P_i$$

### Job Releases



### Super Loop execution scenario

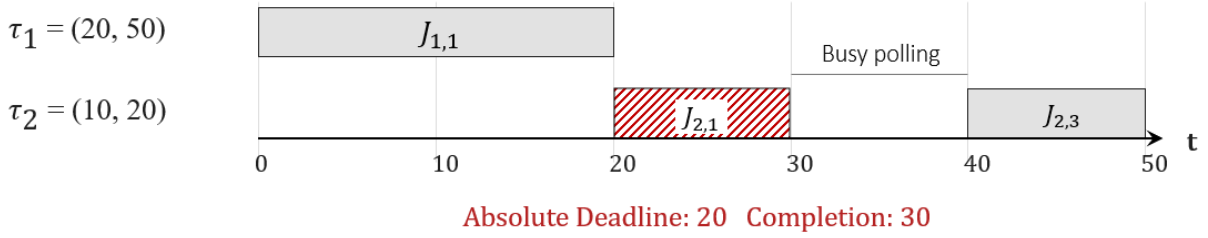


Fig. 2. Super Loop execution scenario for the task set in Table II.

Table I summarizes the notation introduced above.

In the existing schedule figures,  $T_i$ ,  $R_i^j$ , and  $J_i^j$  denote  $P_i$ ,  $r_{i,j}$ , and  $J_{i,j}$ , respectively. The evaluation plots use  $U$  as shorthand for  $U_{\mathcal{T}}$ .

## IV. SCHEDULING PRINCIPLES

### A. Super Loop scheduler

The considered Super Loop scheduler repeatedly scans tasks in a fixed order and executes each selected job non-preemptively. To illustrate its behavior, let us consider the two periodic tasks in Table II. Both tasks first release at  $t = 0$ .

TABLE II  
TASK SET USED IN THE SUPER LOOP EXAMPLE.

| Task     | $C_i$ | $P_i$ | $D_i$ |
|----------|-------|-------|-------|
| $\tau_1$ | 20    | 50    | 50    |
| $\tau_2$ | 10    | 20    | 20    |

The resulting Super Loop schedule is shown in Fig. 2.

Let  $t^{cur}$  denote the current system time and  $t_i^{next}$  the next release time of task  $\tau_i$ . During execution of  $J_{i,j}$ , the release variable  $t_i^{next}$  retains  $r_{i,j}$ . At completion,  $f_{i,j}$  denotes the completion time, and a deadline miss is recorded if  $f_{i,j} > r_{i,j} + D_i$ . The scheduler then advances  $t_i^{next}$  by  $P_i$ . While  $t_i^{next} < f_{i,j}$ , the job with release time  $t_i^{next}$  is counted as skipped and  $t_i^{next}$  is advanced by  $P_i$  again. A release at  $t_i^{next} = f_{i,j}$  is retained.

Algorithm 1 summarizes the illustrated scheduling logic. Since  $r_{i,1} = 0$ , the release index is  $j = 1 + t_i^{next}/P_i$ , including skipped releases. The clock used by NOW starts at zero.

The corresponding scheduling state is shown in Table III.

TABLE III  
SUPER LOOP SCHEDULING STATE FOR FIG. 2.

| $t^{cur}$ | $t_1^{next}$ | $t_2^{next}$ | Deadline miss check | Release skip check               |
|-----------|--------------|--------------|---------------------|----------------------------------|
| 0         | 0            | 0            | —                   | —                                |
| 20        | 50           | 0            | $20 < r_{1,1} + 50$ | $20 < r_{1,2}$                   |
| 30        | 50           | 40           | $30 > r_{2,1} + 20$ | $30 > r_{2,2}$<br>$30 < r_{2,3}$ |
| 40        | 50           | 40           | —                   | —                                |
| 50        | 50           | 60           | $50 < r_{2,3} + 20$ | $50 < r_{2,4}$                   |

In Fig. 2,  $J_{1,1}$  executes from  $t = 0$  to  $t = 20$ . Then,  $J_{2,1}$  executes until  $t = 30$ , missing its deadline at  $t = 20$ . At completion, the job released at  $r_{2,2} = 20$  is counted as skipped and  $t_2^{next}$  advances to  $r_{2,3} = 40$ . Thus, no  $J_{2,2}$  is executed.

### B. Earliest Deadline First

Unlike Super Loop, Earliest Deadline First (EDF) selects jobs according to their absolute deadlines.

We first consider preemptive EDF with immediate preemption. At each scheduling decision, the ready job with the earliest absolute deadline  $r_{i,j} + D_i$  is selected (Section III). If a newly released job has an earlier absolute deadline than the running job, it preempts the running job.

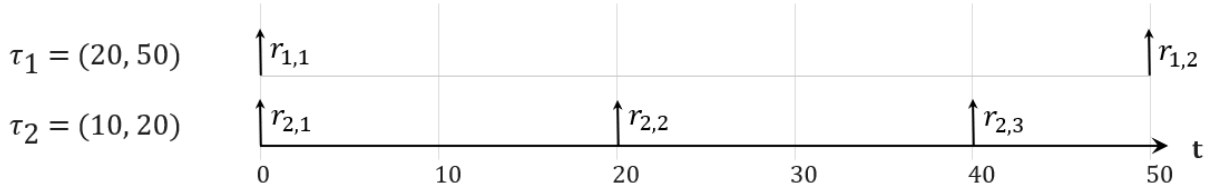
Fig. 3 illustrates EDF scheduling for the task set in Table II.

At  $t = 20$ ,  $J_{2,2}$  is released with  $r_{2,2} + D_2 = 40 < r_{1,1} + D_1 = 50$  and preempts  $J_{1,1}$ . After  $J_{2,2}$  completes at  $t = 30$ ,  $J_{1,1}$  resumes and completes at  $t = 40$ .

Chunked EDF is used as a simplified prototype implementation, avoiding the additional complexity of full task-context switching during initial evaluation. Synthetic jobs execute in chunks of maximum duration  $\Delta$ , with job selection repeated

$$\tau_i = (C_i, D_i, P_i), D_i = P_i$$

### Job Releases



### EDF execution scenario

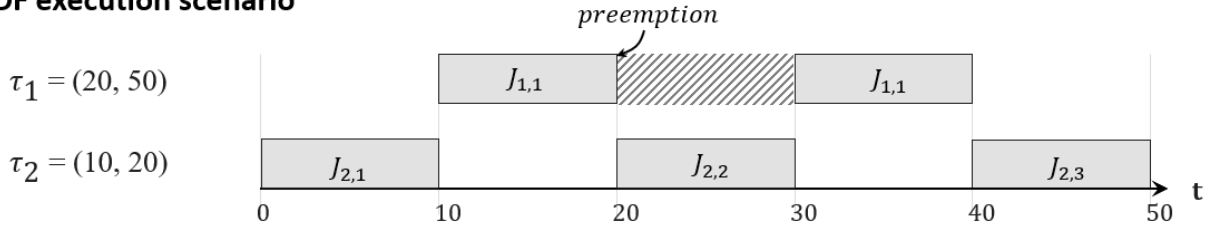


Fig. 3. EDF execution scenario for the task set in Table II. Hatching indicates a preempted job awaiting resumption.

#### Algorithm 1 Super Loop Scheduler

```

1: function SUPERLOOP( $\mathcal{T}$ )
2:   for all  $\tau_i \in \mathcal{T}$  do
3:      $t_i^{next} \leftarrow 0$ 
4:   end for
5:   while true do
6:     for  $i \leftarrow 1$  to  $n$  do ▷ Fixed task order
7:        $t^{cur} \leftarrow \text{NOW}$ 
8:       if  $t^{cur} \geq t_i^{next}$  then
9:          $j \leftarrow 1 + t_i^{next} / P_i$ 
10:         $r_{i,j} \leftarrow t_i^{next}$ 
11:        EXECUTE( $J_{i,j}$ ) ▷ Non-preemptive
12:         $f_{i,j} \leftarrow \text{NOW}$ 
13:        if  $f_{i,j} > r_{i,j} + D_i$  then
14:          RECORDMISS( $J_{i,j}$ )
15:        end if
16:         $t_i^{next} \leftarrow t_i^{next} + P_i$ 
17:        while  $t_i^{next} < f_{i,j}$  do
18:          RECORDSKIP( $\tau_i, t_i^{next}$ )
19:           $t_i^{next} \leftarrow t_i^{next} + P_i$ 
20:        end while
21:      end if
22:    end for
23:  end while
24: end function

```

after each chunk. An earlier-deadline job released during a chunk may therefore wait until the chunk completes. Sensor jobs use separate staged handlers.

For illustration, Fig. 4 uses  $\Delta = 4$  ms. Job  $J_{2,2}$  is released at  $t = 20$  ms and preempts  $J_{1,1}$  at the next chunk boundary  $t = 22$  ms. Thus, the preemption delay is  $\delta = 2$  ms. The

final chunk of  $J_{2,2}$  lasts 2 ms, since the preceding two chunks account for 8 ms of its execution time  $C_2 = 10$  ms.

Ignoring scheduling overhead, the delay until the next chunk boundary satisfies  $0 \leq \delta \leq \Delta$ .

Algorithm 2 summarizes chunked EDF for synthetic jobs, assuming first releases at  $t = 0$  as in the example. The set  $\mathcal{R}$  contains jobs admitted for execution but not yet completed. Initially,  $\mathcal{R} = \emptyset$ ,  $t_i^{next} = 0$ , and no job is active. Equal deadlines are resolved by task order. EXECUTECHUNK executes at most  $\Delta$  of the selected job's remaining execution and preserves its progress.

RELEASEJOB creates the job associated with  $t_i^{next}$ , sets its release and absolute deadline, and adds it to  $\mathcal{R}$  as active. COMPLETEJOB checks its deadline and advances  $t_i^{next}$  using the skip rule in Algorithm 1, then removes the job from  $\mathcal{R}$  and marks the task inactive.

## V. PERFORMANCE EVALUATION

### A. Experimental setup

Experiments run on the NUCLEO-F767ZI at 96 MHz. Physical sensors are disabled in these experiments. Each synthetic job executes a timed busy-wait workload, the scheduler also uses busy polling. Timing is obtained from the DWT cycle counter, and UART reports are emitted after measurement stops.

The results below use three tasks with  $(P_1, P_2, P_3) = (10, 50, 200)$  ms and  $D_i = P_i$ . Table IV lists the configured synthetic workload durations, which are nominal budgets rather than measured worst-case bounds. Thus,  $U_{\mathcal{T}}$  denotes nominal workload utilization and excludes instrumentation and scheduling overhead. Both schedulers use the same task set, Super Loop scans tasks in index order. For clarity, the illustrative schedules assume synchronous first releases at

**Algorithm 2** Chunked EDF Scheduler

---

```

1: function CHUNKEDEDF( $\mathcal{T}, \Delta$ )
2:   while true do
3:      $t^{cur} \leftarrow \text{NOW}$ 
4:     for all  $\tau_i \in \mathcal{T}$  do
5:       if  $t_i^{next} \leq t^{cur}$  and  $\tau_i$  has no active job then
6:         RELEASEJOB( $\tau_i, t_i^{next}$ )
7:       end if
8:     end for
9:     if  $\mathcal{R} \neq \emptyset$  then
10:       $J_{i,j} \leftarrow \text{SELECTEDF}(\mathcal{R})$ 
11:      EXECUTECHUNK( $J_{i,j}, \Delta$ )
12:      if COMPLETED( $J_{i,j}$ ) then
13:         $f_{i,j} \leftarrow \text{NOW}$ 
14:        COMPLETEJOB( $J_{i,j}, f_{i,j}$ )
15:      end if
16:    end if
17:  end while
18: end function

```

---

$t = 0$ , whereas the implementation releases each task for the first time one period after initialization,  $r_{i,1} = t_0 + P_i$ .

TABLE IV

THREE-TASK SYNTHETIC WORKLOADS. EXECUTION BUDGETS ARE IN  $\mu\text{s}$ .

| Nominal $U_{\mathcal{T}}$ (%) | $C_1$ | $C_2$ | $C_3$ |
|-------------------------------|-------|-------|-------|
| 50                            | 1667  | 11111 | 22222 |
| 65                            | 2167  | 14444 | 28889 |
| 75                            | 2500  | 16667 | 33332 |
| 80                            | 2667  | 17778 | 35556 |
| 90                            | 3000  | 20000 | 40000 |
| 95                            | 3167  | 21111 | 42222 |
| 100                           | 3333  | 22222 | 44444 |

The main comparison uses 60 s windows and  $\Delta = 1$  ms for EDF. The chunk-size study uses  $\Delta \in \{1, 2, 4\}$  ms at  $U_{\mathcal{T}} \in \{50, 90, 100\}\%$ , the figure below shows  $U_{\mathcal{T}} = 90\%$ . Each configuration is run three times from reset. Additional campaigns use two tasks and 100 s windows to examine task-set and window-length effects. Extended diagnostic statistics are disabled in all these campaigns.

**B. Evaluation metrics**

Let  $N_i$ ,  $M_i$ , and  $S_i$  be the numbers of completed jobs, completed jobs that missed their deadlines, and skipped releases of  $\tau_i$  in a run. Let  $W$  be the requested measurement-window duration. We report:

- Deadline-miss ratio:  $100M_i/N_i$  percent, defined for  $N_i > 0$ .
- Skipped-release ratio:  $100S_i/(W/P_i)$  percent, normalized by the nominal number of releases in the requested window.
- Average response time: the mean of  $f_{i,j} - r_{i,j}$  over completed jobs. The maximum observed response time is also recorded.
- Selection overhead: measured time spent selecting jobs in iterations that execute work, divided by the actual measurement duration, in percent. This excludes uninstrumented scheduler work and is not total processor utilization.

Measurement stops at the first loop boundary after  $W$ , its actual duration may therefore be slightly longer. Released but unfinished tracked jobs and their overdue status are recorded separately and excluded from completion-based metrics. Subsequent releases awaiting skip accounting are not included in this pending-job count. Plot points are arithmetic means of the corresponding per-run metrics over three runs, not pooled averages over all jobs.

**C. Super Loop and EDF comparison**

Fig. 5 compares the schedulers for  $\tau_1$ . Under Super Loop, its deadline-miss and skipped-release ratios generally increase with utilization, as does its average response time. With chunked EDF ( $\Delta = 1$  ms), no deadline misses or skipped releases are recorded for  $\tau_1$  in the evaluated three-task configurations. Selecting jobs by absolute deadline reduces the blocking caused by the fixed Super Loop order.

Fig. 6 shows the corresponding results for  $\tau_3$ . Its average response time under chunked EDF increases with utilization as jobs with earlier absolute deadlines execute first. At nominal  $U_{\mathcal{T}} = 100\%$ , no capacity is reserved for scheduling and execution overhead, deadline misses and skipped releases are observed for  $\tau_3$ . The lower response time of  $\tau_3$  under Super Loop occurs alongside skipped releases of  $\tau_1$ , so it does not imply better service for the entire task set.

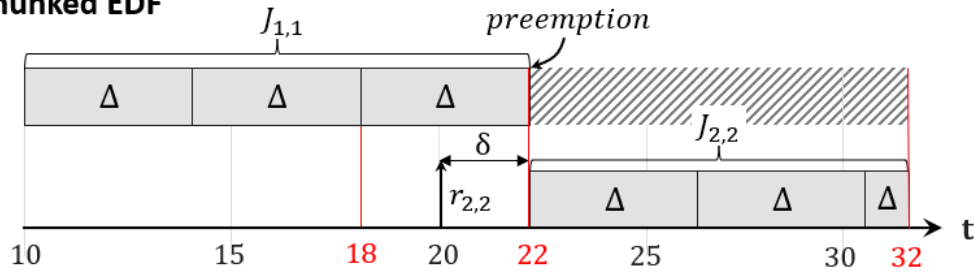
**Chunked EDF**

Fig. 4. Chunk-bounded preemption in the EDF implementation.

#### *D. Effect of chunk duration*

Fig. 7 examines  $\Delta$  at  $U_{\mathcal{T}} = 90\%$ . Larger chunks reduce the frequency of scheduling decisions. In these measurements, the reported selection overhead decreases, while the average response time of  $\tau_1$  increases and that of  $\tau_3$  decreases. Longer non-preemptive intervals delay  $\tau_1$ , the result for  $\tau_3$  also reflects the changed execution order and fewer scheduling interruptions. The overhead measurements remain preliminary pending validation of the instrumentation.

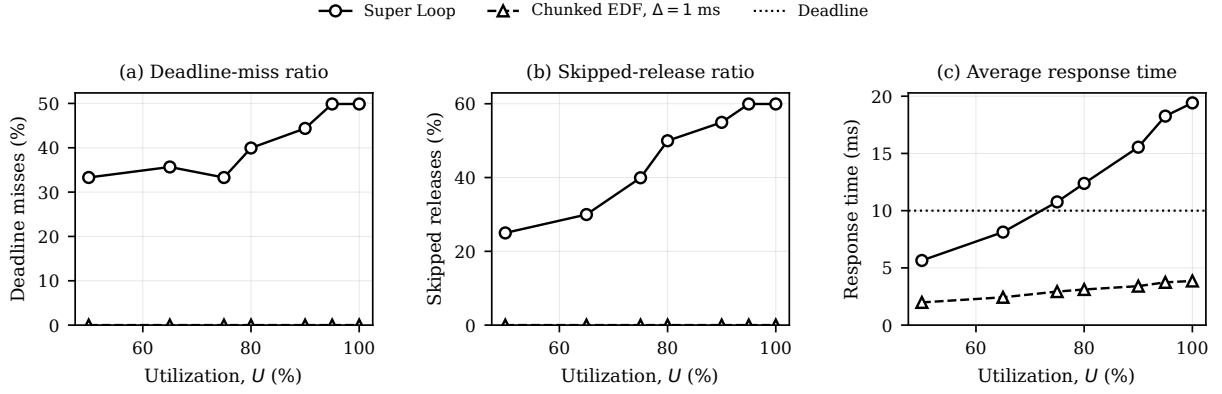


Fig. 5. Performance for  $\tau_1$ : three synthetic tasks,  $\Delta = 1$  ms for EDF, 60 s windows.

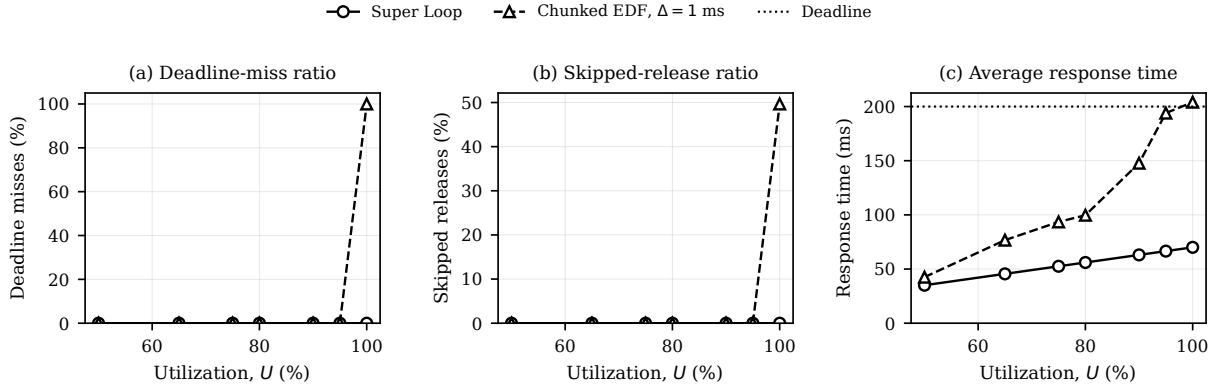


Fig. 6. Performance for  $\tau_3$  under the same conditions as Fig. 5.

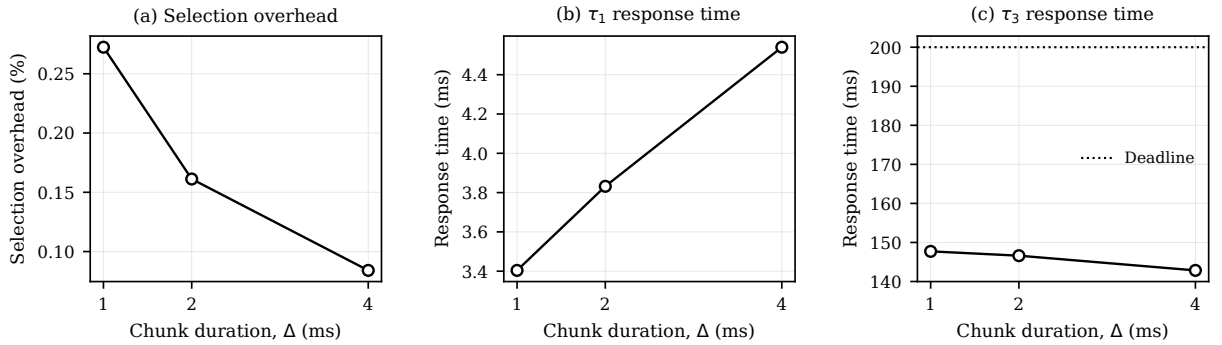


Fig. 7. Impact of chunk duration at  $U_T = 90\%$  with three synthetic tasks and 60 s windows.